

Artificial Intelligence III: Advanced Deep Learning Methods

Ana Neacșu & Vlad Vasilescu

Lecture 1: Attack Mechanisms

National University of Science and Technology POLITEHNICA Bucharest, Romania
BIOSINF Master Program

October 2026

Introduction

Course Overview

- Understanding adversarial attacks on neural networks
- Analyzing vulnerabilities in AI systems
- Exploring defense mechanisms
- Evaluating real-world implications
- Generative Adversarial Neural Networks
- Attention mechanisms
- Transformers and Diffusion-based systems

Course Organization

- Course: **Thursdays at 17:00.**
- **Project discussions after the course.**

Grading (110 points available):

- Student Lectures – **30 points**
- Group Project – **30 points**
- Peer grading reports – **30 points**
- Written Exam – **20 points**

Course details: [GitHub](#).

Course Schedule: October–November 2026

Thursdays at 17:00

Project discussions follow the course.

Date	Session	Topic
8 Oct 2026	0	Intro
15 Oct 2026	1	Adversarial attacks
22 Oct 2026	2	Defense strategies
29 Oct 2026	3	GANs / Generative Models
5 Nov 2026	4	Student Lectures & Group Projects
12 Nov 2026	5	Student Lectures & Group Projects
19 Nov 2026	6	Student Lectures & Group Projects

Courses

Student lectures

Vacations

Evaluations

Times are local to Bucharest.

Course Schedule: November 2026–January 2027

Date	Session	Topic	
26 Nov 2026	7	Student Lectures & Group Projects	
3 Dec 2026	8	Student Lectures & Group Projects	
10 Dec 2026	9	Student Lectures & Group Projects	
17 Dec 2026	10	Student Lectures & Group Projects	
24 Dec 2026	–	Winter break	
31 Dec 2026	–	Winter break	
7 Jan 2027	–	Winter break	
14 Jan 2027	11	Student Lectures & Group Projects	
21 Jan 2027	12	Student Lectures & Group Projects	
To be announced	–	Written Exam Exam period: 23 Jan–12 Feb 2027	
Courses	Student lectures	Vacations	Evaluations

Classes: Thursday, 17:00; project discussions afterwards.

Key Dates and Weekly Deadlines

Choose your topics

- Student lecture topic and date: **15 October 2026, 23:59.**
- Group project topic: **29 October 2026, 23:59.**

Before your assigned Thursday

- **Sunday, 23:59:** lecture outline and bibliography; group project submission before its presentation.
- **Wednesday, 23:59:** student lecture slides in PDF format.
- Send lecture materials to **Ana and Vlad** (email or Teams).

Example: for a lecture on 5 November, the outline is due 1 November and slides are due 4 November.

All deadlines are 23:59, Bucharest time.

Group Project: Attacks and Defenses

- **Teams of up to 2:** choose a classification task and a unique attack/defense configuration. At most two teams per application; register on Teams, first come, first served.
- **Implement the chosen methods yourselves:** no ready-made attack/defense library implementations.
- **Evaluate:** compare clean, attacked and defended performance; benchmark your methods against official implementations.
- **Deliver:** presentation and a documented Git repository with source code, setup/run instructions and method descriptions. No extensive report.
- **Present:** 10–15 minutes, plus 5–10 minutes for questions; include a live or recorded demo.

30 points: presentation 15 + demo 5 + Q&A 10.

Full specification: [GroupProjects.md](#). Submission: Sunday before your project presentation.

Peer Grading Reports

- **Each student:** submit 3 reports on other students' lectures, one per presentation, for up to **30 points**.
- **Participation is required:** attend the lecture and actively engage the presenters with questions.
- Use the [peer grading template](#); give specific, constructive feedback.
- Reports are **anonymous to classmates**. Their scores contribute to the presenting team's peer grading component.
- Email reports to **both Ana and Vlad**, with subject:
`<mm/dd>_<lecture_topic>`

ana_antonia.neacsu@upb.ro vlad.vasilescu2111@upb.ro

Source: [course README](#), Peer Grading Reports.

Student Lectures: Build a Coherent Lesson

Up to 2 students; both speak. 30–35 min + 5–10 min Q&A.

Suggested structure

Motivation and course context → core concepts → methods and comparison
→ evidence, applications and limitations → one clear takeaway.

- Synthesize **5–8 core papers** around a common theme; explain the key ideas in depth and compare assumptions and limitations.
- Use consistent notation, design and technical depth across speakers. Rehearse the timing and involve the audience; a demo is encouraged.
- Put names on the title slide and an extended reading list at the end. Credit borrowed slides, figures and code explicitly.

30 points: outline 5 + slides 5 + peer grade 5 + presentation 15.

Timing penalty: –5 points; missing the signup deadline: 50% penalty.

Topics: [topic list](#). Guidance and grading: [course README](#).

Motivation

Machine Learning – powerful tools used in a wide range of applications

Problem

- Main challenge nowadays: developing high-performance AI systems that are **reliable** and **safe**.
- Modern machine learning systems are vulnerable to **adversarial manipulation** of their inputs.
- Evaluating neural networks robustness against adversarial inputs: **open issue**.

Open questions

- * How safe neural networks are?
- * How can we train robust networks?

Motivation

Machine Learning – powerful tools used in a wide range of applications

Problem

- Main challenge nowadays: developing high-performance AI systems that are **reliable** and **safe**.
- Modern machine learning systems are vulnerable to **adversarial manipulation** of their inputs.
- Evaluating neural networks robustness against adversarial inputs: **open issue**.

Open questions

- * How safe neural networks are?
- * How can we train robust networks?

Adversarial threat

adversarial inputs = original input + adversarial perturbation

Origin of adversarial perturbations

- carefully created with the intention of sabotaging the system
- can arise naturally for several reasons:
 - * Sensor Noise
 - * Ambiguity in Data
 - * Occlusions
 - * Unforeseen Context
 - * Adversarial Intent in Real World
 - * System Limitations

Vulnerabilities of Neural Networks

- **Sensitivity to Input Variations:** Small perturbations to input data can lead to incorrect outputs.
- **Overfitting:** Models might not generalize well to unseen data, leading to poor performance in real-world scenarios.
- **Lack of Transparency:** Their “black box” nature makes them difficult to interpret and debug.

Why Are These Vulnerabilities Important?

- **Trustworthiness:** Users must trust AI systems to make reliable decisions.
- **Security Risks:** Vulnerabilities can be exploited, leading to significant consequences in critical applications.
- **Ethical and Legal Considerations:** Ensuring compliance with regulations and ethical standards is essential.

Identifying Vulnerabilities

- **Model Testing:** Rigorous testing methodologies to uncover weaknesses in models.
- **Adversarial Testing:** Introducing adversarial examples to probe model behavior under stress.
- **Red Team Exercises:** Employing tactics to simulate attacks and understand model limitations.

Importance of Assessing Vulnerabilities

- ☆ **Growing Role of AI:** Neural networks are increasingly applied in critical areas such as security, healthcare, and finance.
- ☆ **Potential Risks:** Adversarial attacks can compromise the reliability and security of AI systems.
- ☆ **Adversarial Robustness:** Ensuring neural network models are robust against malicious inputs is crucial.

Historical Context

- **Early Discoveries:**

- ☆ Initial findings of adversarial vulnerabilities in image recognition models, such as perturbations causing misclassifications.
- ☆ Highlighted the need for developing robust defenses to secure AI models.

- **Impactful Studies:**

- ☆ Landmark research papers such as [Intriguing properties of neural networks](#) by Szegedy et al. (2013), which brought significant attention to adversarial threats.
- ☆ Continuous evolution with studies focusing on defense strategies like adversarial training, randomized smoothing, etc.

- **Generative Models and Adversarial AI:**

- ☆ Models like ChatGPT showcase the power of generative AI to understand and generate human-like text.
- ☆ Rise of adversarial examples and attacks tailored for these generative models, aiming to exploit their behavior and outputs.
- ☆ Necessity to evaluate and mitigate vulnerabilities in generative models to ensure reliability and trustworthiness.

Examples



Model Confidence: 99.7% stop sign



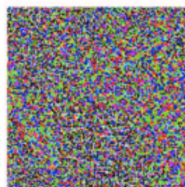
Model Confidence: 0.9 % stop sign

Examples



‘Duck’

+



$\times 0.07$

=



‘Horse’



‘How are you?’

+



$\times 0.01$

=



‘Open the door’

Adversarial attacks

- Naturally occurring perturbations $\rightsquigarrow$ hard to control
- Intentionally perturbing input data $\rightsquigarrow$ erroneous output
- Key feature $\rightsquigarrow$ **input perturbations should be as small as possible**

Several ways of creating adversarial perturbations:

- * **black-box attacks** – the attacker does not have access to the victim model
- * **white-box attacks** – the attacker has access to the victim model
- * **gray-box attacks** – the attacker has limited-access to the victim model

Adversarial attacks

- Naturally occurring perturbations $\rightsquigarrow$ hard to control
- Intentionally perturbing input data $\rightsquigarrow$ erroneous output
- Key feature $\rightsquigarrow$ input perturbations should be as small as possible

Several ways of creating adversarial perturbations:

- * **black-box attacks** – the attacker does not have access to the victim model
- * **white-box attacks** – the attacker has access to the victim model
- * **gray-box attacks** – the attacker has limited-access to the victim model

Adversarial Attacks

Introduction to Adversarial Attacks

Consider a data point $x_0 \in \mathbb{R}^d$, belonging to class $\mathcal{C}_i$.

Adversarial attack $\rightsquigarrow$ a malicious attempt which tries to perturb x_0 to a new data point x such that x is **misclassified** by the classifier.

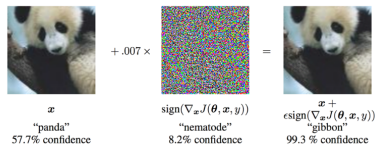


Figure: A classical example by Goodfellow et al 2014. See [this paper](#) for more details.

- **Misconception:** Although often associated with deep neural networks, adversarial attacks are intrinsic to all classifiers due to their tendency to overfit.
- **Focus of Study:** This chapter explores adversarial attacks in linear classifiers to understand their source, geometric considerations, and potential defense strategies.

Understanding Adversarial Attacks

Definition: A malicious attempt to perturb a data point x_0 to another point x such that x belongs to a target adversarial class.

- **Example:** Transforming a feature vector of a *cat* image (x_0) into another feature vector (x) classified as a *dog* or a class specified by the attacker.

Types of Attacks:

- **Targeted Attack:** Aims to move x_0 from its original class C_i to a specific target class C_t .
- **Untargeted Attack:** Seeks to push x_0 away from its original class C_i without a specific target class.

Focus: Initially concentrating on understanding targeted attacks.

Definition 1. (Adversarial attack)

Let $x_0 \in \mathbb{R}^d$ be a data point belonging to class C_i . Define a target class C_t . An adversarial attack is a mapping $\mathcal{A} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ such that the perturbed data

$$x = \mathcal{A}(x_0)$$

is misclassified as C_t , with $t \neq i$.

Additive Adversarial attacks

Among many adversarial attack models, the most commonly used one is the additive model, where we define A as a linear operator that adds perturbation to the input.

Definition 2. (Additive Adversarial Attack)

Let $x_0 \in \mathbb{R}^d$ be a data point belonging to class $\mathcal{C}_i$. Define a target class $\mathcal{C}_t$. An **additive** adversarial attack is an addition of a perturbation $r \in \mathbb{R}^d$ such that the perturbed data

$$x = x_0 + r$$

is misclassified as $\mathcal{C}_t$.

Advantages:

- the input space remains unchanged.
- additive attack allows interpretable analysis with simple geometry.

Formulating Adversarial Attacks

Let $\mathcal{C}_i$ be the true class of x_0 and $\mathcal{C}_t$ which we wish the attack data x to be. Consider a k -class scenario where we have classes $\mathcal{C}_1, \dots, \mathcal{C}_k$. The decision boundaries are specified by k discriminant functions $g_i(\cdot) \quad \forall i = 1, \dots, k$. Then, an adversarial attack problem should satisfy the following:

$$g_t(x) \geq g_j(x), \quad \forall j \neq t \quad (1)$$

Rewriting the $k - 1$ inequalities, we can equivalently express them as:

$$g_t(x) \geq \max_{j \neq t} \{g_j(x)\} \iff \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0. \quad (2)$$

Observations :

- the goal of adversarial attack is to find x such that the inequality in Equation (2) is satisfied.
- The solution is not unique.

Formulating Adversarial Attacks

Definition 3. Minimum Norm Attack

The **minimum norm attack** finds a perturbed data point x by solving the optimization problem:

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad ||x - x_0|| \\ & \text{subject to} \quad \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0, \end{aligned}$$

where $|| \cdot ||$ can be any norm specified by the user.

Definition 4. Maximum Allowable Attack

The **maximum allowable attack** finds a perturbed data point x by solving the optimization problem:

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \max_{j \neq t} \{g_j(x)\} - g_t(x) \\ & \text{subject to} \quad ||x - x_0|| \leq \eta, \end{aligned}$$

where $|| \cdot ||$ can be any norm specified by the user, and $\eta > 0$.

Note that $||x - x_0||$ denotes the magnitude of the perturbation.

Formulating Adversarial Attacks

Definition 3. Minimum Norm Attack

The **minimum norm attack** finds a perturbed data point x by solving the optimization problem:

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad ||x - x_0|| \\ & \text{subject to} \quad \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0, \end{aligned}$$

where $|| \cdot ||$ can be any norm specified by the user.

Definition 4. Maximum Allowable Attack

The **maximum allowable attack** finds a perturbed data point x by solving the optimization problem:

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad \max_{j \neq t} \{g_j(x)\} - g_t(x) \\ & \text{subject to} \quad ||x - x_0|| \leq \eta, \end{aligned}$$

where $|| \cdot ||$ can be any norm specified by the user, and $\eta > 0$.

Note that $||x - x_0||$ denotes the magnitude of the perturbation.

Formulating Adversarial Attacks

Definition 5. Regularization-Based Attack

The **regularization-based attack** finds a perturbed data point x by solving the optimization problem:

$$\underset{x}{\text{minimize}} \quad \|x - x_0\| + \lambda(\max_{j \neq t} \{g_j(x)\} - g_t(x)),$$

where $\|\cdot\|$ can be any norm specified by the user, and $\lambda > 0$ is a regularization parameter.

Which is the best formulation ?

- We can show that for judicious choices for λ and η the three solutions are equivalent.
- We will focus in the next part more on *minimum norm attack*.

Formulating Adversarial Attacks

Definition 5. Regularization-Based Attack

The **regularization-based attack** finds a perturbed data point x by solving the optimization problem:

$$\underset{x}{\text{minimize}} \quad \|x - x_0\| + \lambda(\max_{j \neq t} \{g_j(x)\} - g_t(x)),$$

where $\|\cdot\|$ can be any norm specified by the user, and $\lambda > 0$ is a regularization parameter.

Which is the best formulation ?

- We can show that for judicious choices for λ and η the three solutions are equivalent.
- We will focus in the next part more on *minimum norm attack*.

Geometry perspective of attacks

Geometry of Objective Function

The norm $\|x - x_0\|$ measures a **distance** between x and x_0 . Some possible norms are:

Norms

- ℓ_0 -norm : $\phi(x) = \|x - x_0\|_0$, which gives the most sparse solution.
- ℓ_1 -norm : $\phi(x) = \|x - x_0\|_1$, which is a convex surrogate of the ℓ_0 -norm.
- ℓ_2 -norm : $\phi(x) = \|x - x_0\|_2$, which corresponds to the classic Euclidean distance and it is the most used distance in adversarial settings
- ℓ_∞ -norm : $\phi(x) = \|x - x_0\|_\infty$, which minimizes the maximum element of the perturbation.

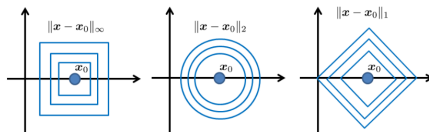


Figure: Geometry of different objective functions

Geometry of the Constraints

we can show that $\Omega = \{x | \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0\}$ is equivalent to

$$\Omega = \left\{ x \mid \begin{array}{l} g_1(x) - g_t(x) \leq 0 \\ g_2(x) - g_t(x) \leq 0 \\ \vdots \\ g_k(x) - g_t(x) \leq 0 \end{array} \right. \quad (3)$$

Depending on the nature of the discriminant function $g_i(x)$, the geometry of Ω could be convex, concave, or arbitrary.

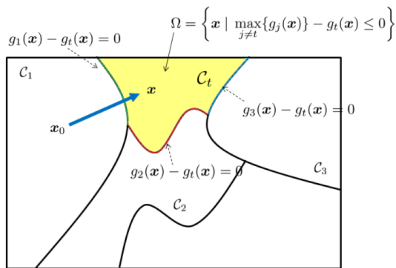


Figure: A typical example constraint set. The decision boundary between classes C_i and C_t is defined by $g_i(x) - g_t(x) = 0$. The decision boundary may change based on the location of x_0 .

Geometry of the Constraints for Linear Classifiers

Consider a k -class linear classifier. Each discriminant function takes the form

$$g_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + w_{i,0}. \quad (4)$$

The decision boundary between the i -th class and the t -th class is therefore

$$g(\mathbf{x}) = (\mathbf{w}_i - \mathbf{w}_t)^\top \mathbf{x} + w_{i,0} - w_{t,0} = 0. \quad (5)$$

Since we want our perturbed data $\mathbf{x}$ to live in $\mathcal{C}_t$, we define the following constraint set

$$\begin{bmatrix} \mathbf{w}_1^\top - \mathbf{w}_t^\top \\ \vdots \\ \mathbf{w}_{t-1}^\top - \mathbf{w}_t^\top \\ \mathbf{w}_{t+1}^\top - \mathbf{w}_t^\top \\ \vdots \\ \mathbf{w}_k^\top - \mathbf{w}_t^\top \end{bmatrix} \mathbf{x} + \begin{bmatrix} w_{1,0} - w_{t,0} \\ \vdots \\ w_{t-1,0} - w_{t,0} \\ w_{t+1,0} - w_{t,0} \\ \vdots \\ w_{k,0} - w_{t,0} \end{bmatrix} \leq \mathbf{0} \Leftrightarrow \mathbf{A}^\top \mathbf{x} \leq \mathbf{b},$$

where $\mathbf{A} = [\mathbf{w}_1 - \mathbf{w}_t, \dots, \mathbf{w}_k - \mathbf{w}_t] \in \mathbb{R}^{d \times (k-1)}$, and $\mathbf{b} = [w_{t,0} - w_{1,0}, \dots, w_{t,0} - w_{k,0}]^\top$

Geometry of the Constraints for Linear Classifiers

Lemma 1 (Constraint Set of Linear Classifier)

Let $g_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + w_{i,0}$ for $i = 1, \dots, k$. We define $A = [\mathbf{w}_1 - \mathbf{w}_t, \dots, \mathbf{w}_k - \mathbf{w}_t] \in \mathbb{R}^{d \times (k-1)}$ and $\mathbf{b} = [w_{t,0} - w_{1,0}, \dots, w_{t,0} - w_{k,0}]^\top$. Then, the constraint set is

$$\Omega = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{A}^\top \mathbf{x} \leq \mathbf{b}\}.$$

Observations:

- the constraint set Ω defines a d -dimensional polytope.
- Ω is convex ??

Corollary: For linear classifiers, the adversarial attack problem is essentially a quadratic minimization:

$$\underset{\mathbf{x}}{\text{minimize}} \|\mathbf{x} - \mathbf{x}_0\| \quad \text{subject to} \quad \mathbf{A}^\top \mathbf{x} \leq \mathbf{b}, \quad (6)$$

which can be solved easily using **convex programming techniques**.

Example

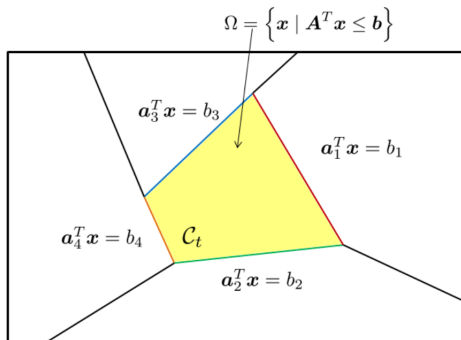


Figure: Geometry of the constraint set Ω for a linear classifier. The constraint set Ω is now a polygon with decision boundaries defined by $a_i^T x = b_i$, where $a_i = w_i - w_t$ and $b_i = w_{i,0} - w_{t,0}$.

Example of ℓ_1 attack

Linear programming formulation for ℓ_1 -norm attack.

Consider minimizing the ℓ_1 -norm, i.e.,

$$\underset{x}{\text{minimize}} \quad \|x - x_0\|_1 \quad \text{subject to} \quad A^\top x \leq b,$$

This problem can be formulated via a linear programming. To do so, we rewrite by letting $r = x - x_0$ so that we have

$$\underset{r}{\text{minimize}} \quad \|r\|_1 \quad \text{subject to} \quad A^\top r \leq \tilde{b},$$

where $\tilde{b} = b - Ax_0$. Now, define r_+ and r_- be the positive and negative parts of r such that $r = r_+ - r_-$. Then, $\|r\|_1 = r_+ + r_-$, and so the optimization problem becomes

$$\underset{r_+, r_-}{\text{minimize}} \quad r_+ + r_-$$

$$\text{subject to} \quad \begin{bmatrix} A^\top & -A^\top \end{bmatrix} \begin{bmatrix} r_+ \\ r_- \end{bmatrix} \leq \tilde{b}, \quad r_+ \geq 0, \quad \text{and} \quad r_- \geq 0.$$

This is a standard linear programming problem, which can be solved efficiently using the Simplex method.

Geometry of the attack

Two important factors:

- the distance metric (e.g. ball, diamond, square)
- the feasible set Ω which specifies the decision boundary between the original and the target class

Theorem 1 (Minimizing ℓ_2 -Norm Attack as a Projection) The adversarial attack

$$\begin{aligned} x^* &= \underset{x \in \Omega}{\operatorname{argmin}} \quad \|x - x_0\|_2, \quad \text{where} \quad \Omega = \{x \mid \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0\}, \\ &= \mathcal{P}_\Omega(x_0) \end{aligned}$$

where $\mathcal{P}_\Omega(\cdot)$ is the projection onto the set Ω .

Geometry of the attack

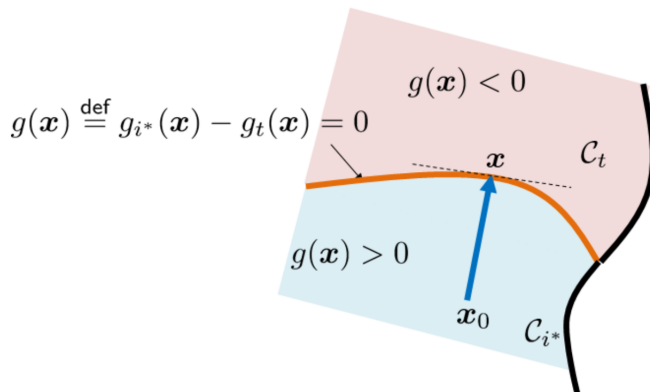


Figure: Given an input data point $\mathbf{x}_0$, our goal is to send $\mathbf{x}_0$ to a targeted class C_t by minimizing the distance between $\mathbf{x}$ and $\mathbf{x}_0$. The decision boundary is characterized by $g(\mathbf{x}) = g_{i^*}(\mathbf{x}) - g_t(\mathbf{x})$. The optimal solution is the projection of $\mathbf{x}_0$ onto the decision boundary.

Parameterize the attack

We can define a step size $\alpha \geq 0$ such that

$$\mathbf{x} = \mathbf{x}_0 + \alpha(\mathcal{P}_\Omega(\mathbf{x}_0) - \mathbf{x}_0), \quad (7)$$

where the residue vector $\mathbf{r} = \mathcal{P}_\Omega(\mathbf{x}_0) - \mathbf{x}_0$ accounts for the direction of the perturbation.

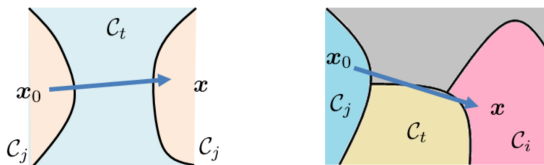


Figure: For inappropriately chosen step size α , the data point x_0 can be sent to a wrong class.

Targeted and Untargeted Attacks

Targeted attack:

- move a data point x_0 to the target class $\mathcal{C}_t$
- define the following constraint:

$$\Omega = \{x \mid \max_{j \neq t} \{g_j(x)\} - g_t(x) \leq 0\}$$

Untargeted attack:

- we like to move x_0 away from its current class, e.g. if $x_0 \in \mathcal{C}_i$, we want $x \notin \mathcal{C}_i$.
- the constraint set of an untargeted attack is given by:

$$\Omega = \{x \mid g_i(x) - \min_{j \neq i} \{g_j(x)\} \leq 0\}.$$

Targeted vs. Untargeted attacks – visual examples

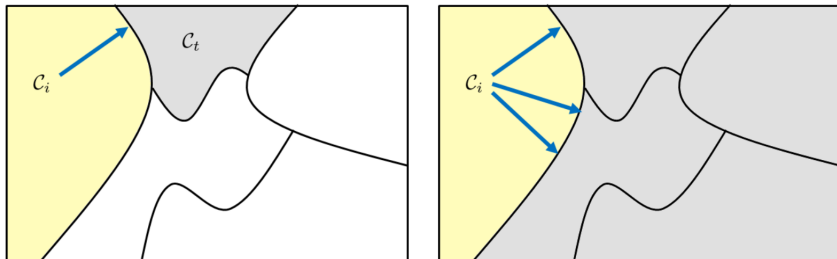


Figure: [left] Targeted attack: The attack has to be specific from $\mathcal{C}_i$ to $\mathcal{C}_t$. [right] Untargeted attack: The attack vector can point to anywhere outside $\mathcal{C}_i$.

White box vs. Black box attacks

White-box attack: assume complete knowledge about the classifier, i.e., we know exactly the discriminant functions $g_i(\mathbf{x})$ for every i .

Black-box attack: we know absolutely nothing about the classifier.

Observation: In this course, in the black-box scenario we will assume we are able to probe the classifier for a fixed number of trials, denoted M . In this case, the constraint set becomes:

$$\Omega = \{\mathbf{x} \mid \max_{j \neq t} \{\tilde{g}_j(\mathbf{x})\} - \tilde{g}_t(\mathbf{x}) \leq 0\},$$

where $\tilde{g}_i$ has been evaluated at $\tilde{g}_i(\mathbf{x}^{(1)}), \tilde{g}_i(\mathbf{x}^{(2)}), \dots, \tilde{g}_i(\mathbf{x}^{(M)})$.

Generating Attacks

Minimum Norm Attack

Let us consider a simple linear classifier with only two classes. The associated discriminant function is defined as

$$g_i(\mathbf{x}) = \mathbf{w}_i^\top \mathbf{x} + w_{i0},$$

where $\mathbf{w}_i \in \mathbb{R}^d$ and $w_{i0} \in \mathbb{R}$. By defining $\mathbf{w} \stackrel{\text{def}}{=} \mathbf{w}_i - \mathbf{w}_t$ and $w_0 \stackrel{\text{def}}{=} w_{i0} - w_{t0}$, we can simplify the discriminant function

$$g_i(\mathbf{x}) - g_t(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0.$$

The adversarial attack can be formulated as

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 \quad \text{subject to} \quad g_i(\mathbf{x}) - g_t(\mathbf{x}) = 0.$$

Theorem 2 (Minimum ℓ_2 Norm Attack for Two-Class Linear Classifier). The adversarial attack to a two-class linear classifier is the solution of

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 \quad \text{subject to} \quad \mathbf{w}^\top \mathbf{x} + w_0 = 0,$$

which is given by

$$\mathbf{x}^* = \mathbf{x}_0 - \left(\frac{\mathbf{w}^\top \mathbf{x}_0 + w_0}{\|\mathbf{w}\|_2} \right) \frac{\mathbf{w}}{\|\mathbf{w}\|_2}.$$

Proof of Theorem 2

Minimum Norm Attack ℓ_2 – Visualisation

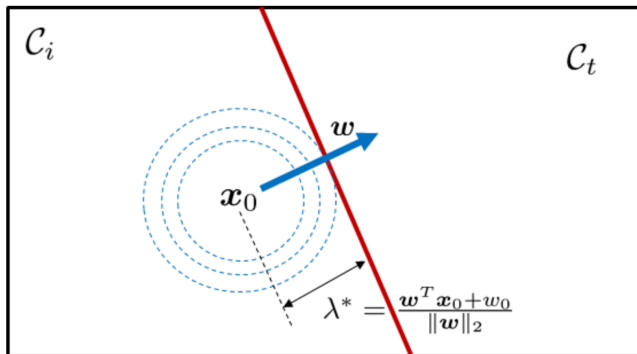


Figure: Geometry of minimum-norm attack for a two-class linear classifier with objective function $\|x - x_0\|_2$. The solution is a projection of the input x_0 onto the separating hyperplane of the classifier.

Minimum Norm Attack – other norms

How can we extend the results to ℓ_∞ ?

Theorem 3 (Hölder's Inequality) Let $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{y} \in \mathbb{R}^d$. Then,

$$-||\mathbf{x}||_p ||\mathbf{y}||_q \leq \mathbf{x}^\top \mathbf{y} \leq ||\mathbf{x}||_p ||\mathbf{y}||_q,$$

for any p and q such that $\frac{1}{p} + \frac{1}{q} = 1$, where $p \in [1, \infty]$.

Consider the following optimization problem:

$$\underset{\mathbf{x}}{\text{minimize}} \quad ||\mathbf{x} - \mathbf{x}_0||_\infty \quad \text{subject to} \quad \mathbf{w}^\top \mathbf{x} + w_0 = 0.$$

Minimum Norm Attack – other norms

Theorem 4 (Minimum ℓ_∞ Norm Attack for Two-Class Linear Classifier) The minimum ℓ_∞ norm attack for a two-class linear classifier, i.e.,

$$\underset{x}{\text{minimize}} \quad \|x - x_0\|_\infty \quad \text{subject to} \quad w^\top x + w_0 = 0$$

is given by

$$x = x_0 - \left(\frac{w^\top x_0 + w_0}{\|w\|_1} \right) \cdot \text{sign}(w).$$

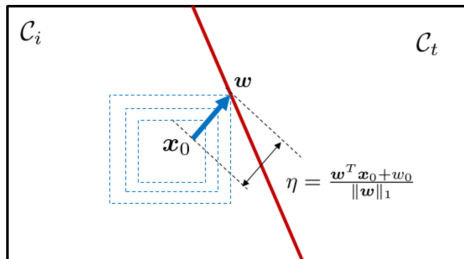


Figure: Geometry of minimum ℓ_∞ norm attack for a two-class linear classifier with objective function $\|x - x_0\|_\infty$

DeepFool Attack

- Introduced by Moosavi-Dezfooli et al. in 2016
- Check [here](#) the original paper; an [improved version](#) of the attack was proposed in 2023.
- It is a generalization of the minimum ℓ_2 -norm attack.

Definition 6 (DeepFool Attack)

The DeepFool attack for a two-class classification generates the attack by solving the optimization problem

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 \quad \text{subject to} \quad g(\mathbf{x}) = 0,$$

where $g(\mathbf{x}) = 0$ is the nonlinear decision boundary separating the two classes.

DeepFool Algorithm

Problem: Due to the fact that $g(\mathbf{x})$ is non-linear $\rightarrow$ very difficult to derive a closed-form expression.

Solution: Compute the solution iteratively, using first order approx. of $g(\mathbf{x})$

$$g(\mathbf{x}) \approx g(\mathbf{x}^{(k)}) + \nabla_{\mathbf{x}} g(\mathbf{x}^{(k)})^\top (\mathbf{x} - \mathbf{x}^{(k)}),$$

where $\mathbf{x}^{(k)}$ is the k -th iterate of the solution.

Corollary 1. (DeepFool Algorithm for Two-Class Problem)

An iterative procedure to obtain the DeepFool attack solution is

$$\begin{aligned} \mathbf{x}^{(k+1)} &= \underset{\mathbf{x}}{\operatorname{argmin}} \quad \|\mathbf{x} - \mathbf{x}^{(k)}\|_2 \quad \text{s. t.} \quad g(\mathbf{x}^{(k)}) + \nabla_{\mathbf{x}} g(\mathbf{x}^{(k)})^\top (\mathbf{x} - \mathbf{x}^{(k)}) = 0 \\ &= \mathbf{x}^{(k)} - \left(\frac{g(\mathbf{x}^{(k)})}{\|\nabla_{\mathbf{x}} g(\mathbf{x}^{(k)})\|_2} \right) \nabla_{\mathbf{x}} g(\mathbf{x}^{(k)}). \end{aligned}$$

Question: How it will extend to multi-class?

Maximum-Allowable Attack

We will consider the case of maximum allowable ℓ_∞ attack. The optimization problem can be formulated as follows.

$$\underset{x}{\text{minimize}} \quad w^\top x + w_0 \quad \text{subject to} \quad \|x - x_0\|_\infty \leq \eta.$$

Theorem 5 (Maximum Allowable ℓ_∞ Norm Attack of Two-Class Linear Classifier) The maximum allowable ℓ_∞ norm attack for a two-class linear classifier, i.e.,

$$\underset{x}{\text{minimize}} \quad w^\top x + w_0 \quad \text{subject to} \quad \|x - x_0\|_\infty \leq \eta.$$

is given by

$$x = x_0 - \eta \cdot \text{sign}(w).$$

Question: How it will extend to ℓ_2 norm?

Fast Gradient Sign Method (FGSM)

Idea: maximize certain loss function $J(\mathbf{x}; \mathbf{w})$, subject to an upper bound on the perturbation, e.g., $\|\mathbf{x} - \mathbf{x}_0\|_\infty \leq \eta$.

Definition 7 (Fast Gradient Sign Method (FGSM) by Goodfellow et al 2014)

Given a loss function $J(\mathbf{x}; \mathbf{w})$, the FGSM creates an attack $\mathbf{x}$ by

$$\mathbf{x} = \mathbf{x}_0 + \eta \cdot \text{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})),$$

where $\nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})$ should be interpreted as the gradient of J with respect to $\mathbf{x}$ evaluated at $\mathbf{x}_0$

Question: How do we specify the loss?

For binary classification we can derive an expression.

Example. (FGSM Loss Function for a Two-Class Linear Classifier)

Recall that the objective function in the maximum allowable attack is

$$\varphi(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = (\mathbf{w}_i^\top \mathbf{x} + w_{i0}) - (\mathbf{w}_t^\top \mathbf{x} + w_{t0})$$

We define a loss function $J(\mathbf{x}) = (\mathbf{w}_i^\top \mathbf{x} + w_{i0}) - (\mathbf{w}_t^\top \mathbf{x} + w_{t0})$.

If $J(\mathbf{x}) \geq 0$, then $\mathbf{x}$ is misclassified. We can define the following objective

$$J(\mathbf{x}; \mathbf{w}) = -(\mathbf{w}^\top \mathbf{x} + w_0).$$

FGSM as an optimization problem

For general, non-linear loss functions, we can use first order approximation.

$$J(\boldsymbol{x}; \boldsymbol{w}) = J(\boldsymbol{x}_0 + \boldsymbol{r}; \boldsymbol{w}) \approx J(\boldsymbol{x}_0; \boldsymbol{w}) + \nabla_{\boldsymbol{x}} J(\boldsymbol{x}_0; \boldsymbol{w})^\top \boldsymbol{r}.$$

Corollary 2 (FGSM as a Maximum Allowable Attack Problem).

The FGSM attack can be formulated as the optimization with $J(\boldsymbol{x}; \boldsymbol{w})$ being the loss function:

$$\underset{\boldsymbol{r}}{\text{maximize}} \quad \nabla_{\boldsymbol{x}} J(\boldsymbol{x}_0; \boldsymbol{w})^\top \boldsymbol{r} + J(\boldsymbol{x}_0; \boldsymbol{w}) \quad \text{subject to} \quad ||\boldsymbol{r}||_\infty \leq \eta,$$

of which the solution is given by

$$\boldsymbol{x} = \boldsymbol{x}_0 + \eta \cdot \text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{x}_0; \boldsymbol{w})).$$

Observation: Knowing that FGSM corresponds to a maximum allowable attack with ℓ_∞ norm, we can easily generalize the attack to other ℓ_p norms.

Iterative Gradient Sign Method

- Introduced by Kurakin et. al. in 2017
- Addresses the problem of the unboundedness of x

$$\begin{aligned} \underset{x}{\text{maximize}} \quad & \nabla_x J(x_0; w)^\top x - \nabla_x J(x_0; w)^\top x_0 + J(x_0; w) \\ \text{subject to} \quad & \|x - x_0\|_\infty \leq \eta, \quad 0 \leq x \leq 1. \end{aligned}$$

Corollary 4 (I-FGSM Algorithm as Projected FGSM).

The Iterative FGSM algorithm generates the attack by iteratively solving

$$x^{(k+1)} = \underset{0 \leq x \leq 1}{\operatorname{argmax}} \nabla_x J(x^{(k)}; w)^\top x \quad \text{subject to} \quad \|x - x^{(k)}\|_\infty \leq \eta,$$

of which the per-iteration solution is given by

$$x^{(k+1)} = \mathcal{P}_{[0,1]} \{x^{(k)} + \eta \cdot \operatorname{sign}(\nabla_x J(x^{(k)}; w))\},$$

where $\mathcal{P}_{[0,1]}(x)$ is a projection operator that elementwisely projects out of bound values to the bound $0 \leq x \leq 1$.

FGSM Function

```
1 def fgsm(model, X, y, epsilon):
2     """ Construct FGSM adversarial examples on the examples X"""
3     delta = torch.zeros_like(X, requires_grad=True)
4     loss = nn.CrossEntropyLoss()(model(X + delta), y)
5     loss.backward()
6     return epsilon * delta.grad.detach().sign()
```

Regularization-based Attack

For advanced classifiers such as deep neural networks, solving an optimization involving constraints are typically very difficult.

The regularization-based attack considers the problem

$$\underset{x}{\text{minimize}} \quad \|x - x_0\|_2 + \lambda \left(\max_{j \neq t} \{g_j(x)\} - g_t(x) \right).$$

In the case of binary classification, we use the following simplification

$$\underset{x}{\text{minimize}} \quad \|x - x_0\|_2 + \lambda(w^\top x + w_0).$$

Theorem 6 (Regularization-based Attack for Two-Class Linear Classifier) The regularization-based attack for a two-class linear classifier generates the attack by solving

$$\underset{x}{\text{minimize}} \quad \frac{1}{2} \|x - x_0\|_2 + \lambda(w^\top x + w_0),$$

of which the solution is given by

$$x = x_0 - \lambda w.$$

Carlini & Wagner

- Proposed by Carlini et. al. in 2016
- It is a modified regularization based-attack to address the unboundedness of the objective.
- The optimization can be formulated as

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 + \iota_\Omega(\mathbf{x}), \quad (8)$$

where

$$\iota_\Omega(\mathbf{x}) = \begin{cases} 0, & \text{if } \max_{j \neq t} \{g_j(\mathbf{x})\} - g_t(\mathbf{x}) \leq 0, \\ +\infty, & \text{otherwise.} \end{cases} \quad (9)$$

Carlini-Wagner attack relaxes ι_Ω by considering a rectifier function

$$\zeta(x) = \max(x, 0)$$

The optimization problem becomes

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 + \lambda \zeta(\mathbf{x}),$$

for $\lambda > 0$.

Carlini & Wagner

The attack can be written as

$$\underset{\mathbf{x}}{\text{minimize}} \|\mathbf{x} - \mathbf{x}_0\|_2 + \lambda \max \left\{ \left(\max_{j \neq t} \{g_j(\mathbf{x})\} - g_t(\mathbf{x}) \right), 0 \right\}$$

Remarks:

- Other operators besides the rectifier function can be used.
- C&W is convex: $h(\mathbf{x}) = \max(\varphi(\mathbf{x}), 0)$ is convex if φ is convex

Algorithm. (CW Attack Gradient Descent.)

The Gradient Descent algorithm for generating CW attack is given by the following iteration for $k = 1, 2, \dots$:

$$\begin{aligned} i^* &= \underset{j \neq t}{\operatorname{argmax}} \{g_j(\mathbf{x}^k)\} \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \alpha \nabla \varphi(\mathbf{x}^k; i^*). \end{aligned}$$

- α – Gradient descent step size, which controls the rate of convergence
- λ – Regularization parameter, which controls the relative strength between the distance term and the constraint term.

Random Noise Attack

Idea: we perturb the data by pure i.i.d. Gaussian noise.

$$\mathbf{x} = \mathbf{x}_0 + \sigma_r \mathbf{r},$$

where $\mathbf{r} \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

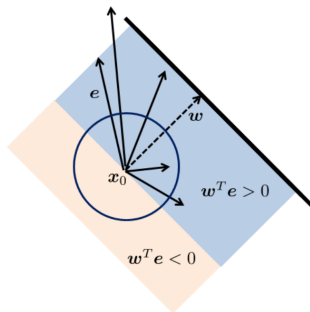


Figure: Attacking the linear classifier with i.i.d. noise is equivalent to putting an uncertainty circle around $\mathbf{x}_0$ with radius σ_r .

Curse of dimensionality: the probability of $\mathbf{w}^\top \mathbf{r} > 0 \rightarrow 0$ as the dimensionality of $\mathbf{r}$ grows.

Random Noise Attack

Let us evaluate the probability of $\mathbf{w}^\top \mathbf{r} \geq \epsilon$ for some $\epsilon > 0$. To this end, let us consider

$$\mathbb{P} \left[\frac{1}{d} \mathbf{w}^\top \mathbf{r} \geq \epsilon \right] = \mathbb{P} \left[\frac{1}{d} \sum_{j=1}^d w_j r_j \geq \epsilon \right],$$

where d is the dimensionality of $\mathbf{w}$, i.e., $\mathbf{w} \in \mathbb{R}^d$ and ϵ is the tolerance.

Theorem 7. Let $\mathbf{w}$ be the weight vector of a linear classifier, and let $\mathbf{x}_0 \in \mathbb{R}^d$ be an input data point. Suppose we attack the classifier by adding i.i.d. Gaussian noise $\mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to $\mathbf{x}_0$. The probability of a successful attack against the classifier with a tolerance level ϵ is bounded by

$$\mathbb{P} \left[\frac{1}{d} \sum_{j=1}^d w_j r_j \geq \epsilon \right] \leq \frac{\|\mathbf{w}\|}{\epsilon d \sqrt{2\pi}} \exp \left\{ -d^2 \frac{\epsilon^2}{2\|\mathbf{w}\|_2} \right\}.$$

Therefore, as $d \rightarrow \infty$ it becomes increasingly more difficult for i.i.d. Gaussian noise to succeed in attacking.

Other resources

[Here](#) you can find a list with most popular attacks.

Modern Attacks

- One Pixel Attack for Fooling Deep Neural Networks
- Augmented Lagrangian Method Attack (ALMA)
- Decoupling Direction and Norm Attack (DDN)

For more information about the world of adversarial attacks check out Nicolas Carlini's [blog](#).